

Step 06 — Dynamic fields API (manual test guide)

Goal: Confirm tenant-scoped field definition CRUD (create, list, update, delete) with RBAC, validation, and audit logging.

Maps to: Phase 2 — Dynamic Field Engine (API only; UI is Step 07)

Do not start Step 07 until every required item passes.

Artifacts:

- Markdown: docs/manual-test-guides/step-06-dynamic-fields-api.md
- PDF: docs/manual-test-guides/step-06-dynamic-fields-api.pdf
- Postman: docs/postman/FMS-API.postman_collection.json (Fields folder)

Prerequisites

- ☐ Steps 01–05 already passing
- ☐ PostgreSQL running; migrations applied (`npm run prisma:migrate` , includes `step06_dynamic_fields`)
- ☐ Super Admin bootstrap env set in `server/.env.local`
- ☐ At least one company with a **company admin** (Step 03)
- ☐ A self-registered **normal user** exists (Step 02 register)

Setup commands

Server:

```
cd finance-management/server
npm install
npx prisma generate
npm run prisma:migrate
npm run dev
```

Client (optional for this step — API-only):

```
cd finance-management/client
npm install
npm run dev
```

Supported field types (MVP)

TEXT , LONG_TEXT , NUMBER , CURRENCY , DATE , BOOLEAN , DROPDOWN , FILE

Targets: EXPENSE , INCOME (definitions are scoped per target within a company)

Checklist

A. Authorization

#	Test	Expected	Pass?
A1	GET /api/v1/fields as company admin	HTTP 200; fields array	[]
A2	Same as normal user	HTTP 403 FORBIDDEN	[]
A3	Same as Super Admin (no tenant)	HTTP 403 TENANT_REQUIRED	[]
A4	Same without cookie	HTTP 401	[]

Login as company admin first (Postman **Auth** → **Login (company admin)**) or curl with cookies).

Example:

```
curl -s -b cookies.txt "http://localhost:4000/api/v1/fields?target=EXPENSE" | jq
```

B. Create field definitions

#	Test	Expected	Pass?
B1	POST text field for EXPENSE	HTTP 201; field.key, field.label, fieldType: TEXT	[]
B2	POST dropdown without options	HTTP 400 validation error	[]
B3	POST dropdown with options.choices	HTTP 201; options.choices returned	[]
B4	POST duplicate key for same target	HTTP 409 FIELD_KEY_TAKEN	[]
B5	Omit key — auto-generated from label	HTTP 201; slug-like key	[]

Create text field:

```
curl -s -b cookies.txt -X POST "http://localhost:4000/api/v1/fields" \
-H "Content-Type: application/json" \
-d '{"target":"EXPENSE","label":"Project Code","fieldType":"TEXT","required":true}' | jq
```

Create dropdown:

```
curl -s -b cookies.txt -X POST "http://localhost:4000/api/v1/fields" \
-H "Content-Type: application/json" \
-d '{"target":"EXPENSE","key":"status","label":"Status","fieldType":"DROPDOWN","options":{"choices":["Draft","Approved"]}}' | jq
```

C. List and read

#	Test	Expected	Pass?
C1	GET /fields?target=EXPENSE	Only expense fields; sorted by sortOrder	[]
C2	GET /fields/:id for own field	HTTP 200; full field object	[]

C3	GET /fields/:id for non-existent id	HTTP 404	[]
----	-------------------------------------	----------	-----

D. Update and delete

#	Test	Expected	Pass?
D1	PATCH label + sortOrder	HTTP 200; updated values	[]
D2	PATCH enabled: false	Field disabled; still listed unless ?enabled=true	[]
D3	DELETE field	HTTP 200 { ok: true }; GET returns 404	[]

Example update:

```
curl -s -b cookies.txt -X PATCH "http://localhost:4000/api/v1/fields/<fieldId>" \
-H "Content-Type: application/json" \
-d '{"label":"Project code","sortOrder":10,"visibleToNormalUser":true}' | jq
```

E. Tenant isolation

#	Test	Expected	Pass?
E1	Company admin lists fields	Only their company's definitions	[]
E2	Company admin GET/PATCH/DELETE another company's field id	HTTP 404	[]

Use two companies with different admins if available.

F. Audit trail

After create, update, and delete, list audit logs as company admin:

#	Test	Expected	Pass?
F1	Create field	CREATE / FieldDefinition; newValues has label/type	[]
F2	Update field	UPDATE / FieldDefinition; oldValues + newValues	[]
F3	Delete field	DELETE / FieldDefinition; oldValues present	[]

```
curl -s -b cookies.txt "http://localhost:4000/api/v1/audit/logs?limit=20" | jq
```

G. RBAC permission map

#	Test	Expected	Pass?
G1	GET /api/v1/rbac/me as company admin	permissions includes fields:manage	[]
G2	Same as normal user	permissions does not include fields:manage	[]

Pass criteria for Step 06

- ☐ All **required** checklist items above pass
- ☐ Postman **Fields** folder requests pass when run as company admin
- ☐ No cross-tenant field access
- ☐ Field mutations appear in audit logs

When all required items pass: mark related Notion tasks Done, then start **Step 07 — Dynamic fields UI**.

Quick Postman order

1. Auth → Login (Super Admin) → Tenants → Create tenant + company admin (if needed)
2. Auth → Login (company admin)
3. Fields → Create text field → Create dropdown → List fields → Update field → Delete field
4. Auth → Login (normal user) → Fields → List fields (expect 403)
5. Audit → List audit logs (confirm field entries)

`docs/manual-test-guides/step-06-dynamic-fields-api.pdf`